

A Slack environment for testing agents

- The assignment asked for a working Slack-like app, agent tasks, and recorded tests.
- I needed a harness that connects model APIs to the app and manages each test run.

01 · RESEARCH



Computer-use interfaces

What can the model see and do?

02 · BUILD



Controlled test runs

How do I execute and verify actions?

03 · COMPARE



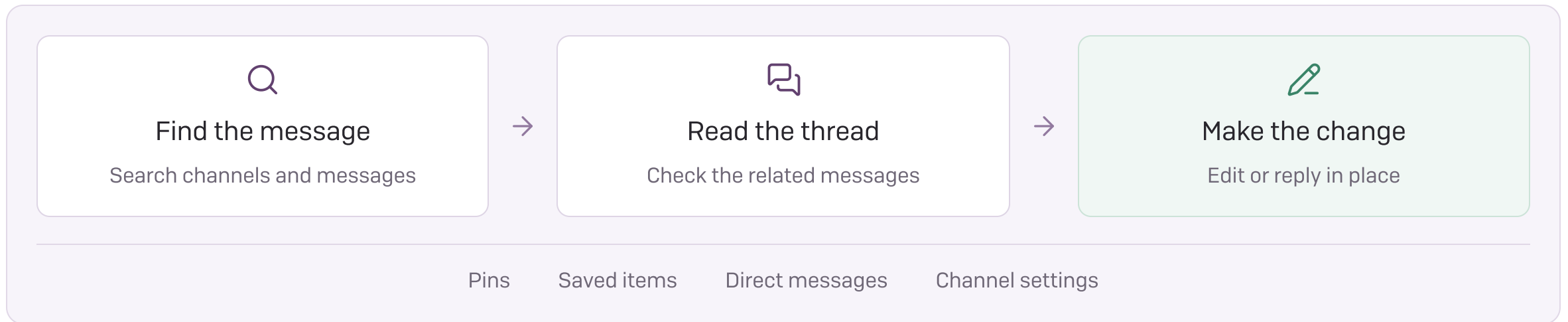
Model performance

Which tasks can each model complete?

-
- Kevin Liu · CUA implementation exercise · October 2026 · Based on the supplied assignment brief.

The app supports complete Slack workflows

- I started with search, threads, and message editing from the assignment.
- Each workflow includes the interface, stored data, and checks for the required result.
- People and browser agents use the same app with fictional workspace data.



- React UI · SQLite state · [Try the workspace without a key](#) · [Prior-work review](#).

The task set covers 18 workflows

Six focused tasks

Update a topic

Channel → edit topic → save

Reply in a thread

Find request → open thread → reply

Edit a message

Find own message → edit → save

Triage an incident

Find active incident → react → pin

Delete a draft

Find own draft → delete → confirm

Send a handoff

Read current details → compose DM

Coordinate work

Coordinate a release *

Read sources → edit → reply → topic → DM

Close out an incident

Read resolution → replace pin → reply → topic

Refresh saved work

Find current sources → replace saved items → DM

Record a design decision

Read approval → react → pin → save → details

Repair an outdated handoff

Read correction → edit DM → react → save

Complete a QA sign-off

Read test results → reply → fix reaction → save

Combine several changes

Publish a release update

Read source → format update → pin → delete draft

Brief the on-call engineer

Read rotation → DM primary → save → details

Correct a threaded estimate

Read review → edit own reply → react → reply

Refresh the launch board

Find plan → replace pin → save → topic

Hand off an approved design *

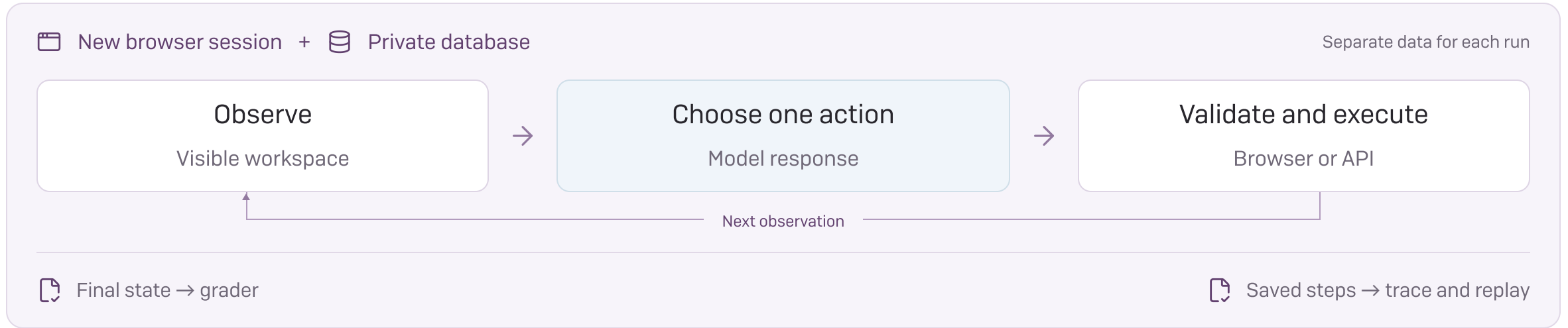
Read decision → react → DM → reply → save

Prepare a release retrospective

Read three sources → summarize → pin → save → DM

- These diagrams summarize the intended workflows. Exact instructions and preservation checks remain in each task.
- * Two original tasks contain wording problems. Their results remain unchanged and are discussed later.

The harness runs the model's actions in a loop



- Each run starts with a separate database and browser session containing known workspace data.
 - The model receives an observation and returns one action at a time.
 - The harness repeats this loop until the model finishes or a limit stops the run.
-
- Playwright controls the browser. Separate data does not protect the host from malicious code.

The model can use four interfaces to the same app

Interface	Observation → action	Interaction
 Pixels	Screenshot → mouse and keyboard	Browser
 Accessibility	Named page controls → click or fill	Browser
 Page JSON	Structured page controls → click or fill	Browser
 Actor API	Workspace data → direct changes	Tool use

- The interface determines what the model observes and which actions it can request.
- The actor API uses the same application rules but does not operate the browser.
- All four paths change workspace data that the same grader can check.

-
- Page JSON describes page controls, not the underlying database. Actor API access exposes broader structured data.
 - The harness uses a shared JSON action protocol, not each provider's native computer-use protocol.

The grader checks the final workspace state

- Workspace state is the stored data: messages, threads, topics, and saved items.
- The grader is code that compares this data with the task requirements.
- A model's claim that it finished does not affect the result.



Example: update a channel topic

Set the requested topic. Make no other changes.



Correct topic

Exact text in channel settings



No extra changes

Other workspace data stays the same

Both pass

Reward = 1

- The grader checks the final state. It does not penalize a wrong action that the agent fully undoes.
- Task definitions and result checks.

I tested the app and grader before comparing models

APP BEHAVIOR



174 backend tests

91 browser tests

Does the software work?

RESULT CHECKS



84 valid states

2,583 invalid states

Does the grader reject wrong results?

RECORDING INTEGRITY



Actions and states

Timing and usage

Can each result be inspected?

- Scripted browser tests complete the workflows without a model.
- Known valid and invalid states test the grader's rules.
- Saved actions and workspace states support trace review and replay without new model calls.

• [Software verification](#) • Counts shown here come from saved full-suite reports, not the later targeted release checks.

The reusable harness made model comparisons possible

- Once the harness worked, I could change the model without rebuilding the test.
- I expanded the assignment into a benchmark prototype with two separate comparisons.
- The first compares models using accessibility controls, and the second compares four interfaces.

Original assignment



Working environment

Focused Slack workflows
Known starting state



Repeatable harness

Reset · act · check · record
Select a model



Benchmark prototype

306 model-comparison runs
96 interface-comparison runs

Expanded scope

The next slides cover the test conditions, results, and limitations of each comparison.

Comparison 1: 17 models, using accessibility controls

$$\begin{array}{ccccccc} \text{🗃️} & & \text{📋} & & \text{🛡️} & & \\ 17 & \times & 18 & \times & 1 & = & 306 \\ \text{model routes} & & \text{tasks} & & \text{attempt per pair} & & \text{recorded attempts} \end{array}$$

- Each model receives named page controls and returns one JSON action per request.
- All 306 runs use the same starting data, recent-four history, and no site guide.
- This tests model performance under one interface, not the best interface for each model.

40 actions 180 s per run \$5 estimated allowance per run

- Seed 1042 • 4,096 output tokens per request • identical graders • provider-default reasoning.
- I kept 105 earlier runs and added 201 new runs without replacing failures.
- [Node 24 setup and API key instructions](#) • [Exact routes, limits, and run history](#). Provider routing and collection time were not controlled.

Results for 17 models

1 per task/model · Collection complete

306 / 306 attempted · 152 passed · 61 incomplete · 93 blocked

\$189.3110

Usage estimate

\$3.7794

Unresolved · 35 calls

\$193.0904

Total allowance · 306 trials

Model ↕	Trials ↕	Passed ↕	Incomplete ↕	Blocked ↕	Median time ↕	Usage estimate ↕	Unresolved ↕	Total allowance ↕
 GPT-6 Astra	18 / 18	14 / 18	1	3	27.1s	\$35.9623	\$0.0000	\$35.9623
 GPT-6.1 Sol	18 / 18	14 / 18	4	0	28.5s	\$8.4352	\$0.0000	\$8.4352
 GPT-6 Luna	18 / 18	9 / 18	8	1	39.4s	\$0.5394	\$0.0092	\$0.5486
 GPT-OSS 120B	18 / 18	5 / 18	2	11	180.1s	\$0.6682	\$0.1358	\$0.8040
 Claude Fable 5.1	18 / 18	13 / 18	1	4	62.8s	\$48.4714	\$0.0000	\$48.4714
 Claude Opus 5.5	18 / 18	13 / 18	4	1	38.3s	\$28.6987	\$0.3741	\$29.0728
 Claude Sonnet 5.5	18 / 18	7 / 18	4	7	39.9s	\$13.9323	\$0.3304	\$14.2627
 Claude Haiku 4.5	18 / 18	6 / 18	3	9	28.3s	\$3.7921	\$0.0000	\$3.7921
 Grok 4.7	18 / 18	10 / 18	6	2	34.3s	\$11.9101	\$0.3809	\$12.2910
 Qwen 3.8 Max	18 / 18	7 / 18	1	10	125.1s	\$6.9999	\$1.0639	\$8.0638
 DeepSeek V4 Pro	18 / 18	8 / 18	1	9	106.8s	\$4.6816	\$0.2547	\$4.9363
 DeepSeek V4.1 Flash	18 / 18	7 / 18	3	8	65.9s	\$1.3825	\$0.0541	\$1.4365
 GLM 5.3	18 / 18	7 / 18	0	11	125.3s	\$3.6307	\$0.3453	\$3.9760
 GLM 5.3 Flash	18 / 18	8 / 18	0	10	46.2s	\$0.4444	\$0.0000	\$0.4444
 Kimi K3	18 / 18	11 / 18	3	4	50.0s	\$14.0051	\$0.7958	\$14.8009
 MiniMax M3	18 / 18	5 / 18	10	3	25.9s	\$1.4698	\$0.0353	\$1.5052
 Nemotron 3 Ultra	18 / 18	8 / 18	10	0	50.1s	\$4.2874	\$0.0000	\$4.2874

Passed Incomplete Blocked Not run

- 18 public tasks. 1 run per task and model. All use accessibility controls. Median time includes blocked runs.
- Total allowance adds the usage estimate to money reserved for missing usage. These are estimates, not invoices. Including earlier tests: \$194.1058 of the \$300 budget. [Full costs CSV](#) · [JSON](#)
- 105 earlier runs are included. No retries. One run per task cannot establish a reliable model ranking. Two tasks have unclear instructions. [Read the failure analysis.](#)
- [Test plan](#) · [Recorded runs](#) · [Download all trials \(CSV\)](#)

Low scores had different causes

- Reviewing the failed tasks exposed two instruction problems that grader tests had not caught.
- Other runs failed because of incorrect actions or limits, so these scores need different explanations.

TASK TEXT

“DESIGN”

No instruction to replace this word

RESULT CHECK

≠

“Willow”

Grader requires this exact name

Release sync

Unclear source location

0 / 17 passed

Retrospective

Every run reached a limit

0 / 17 passed

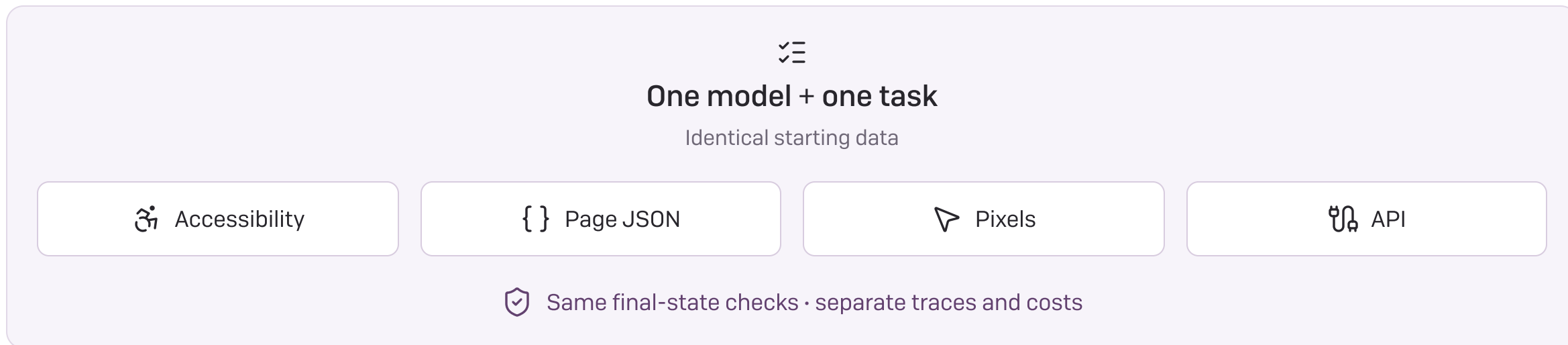
Handoff repair

Sol and Astra clicked Cancel

5 / 17 passed

- Six completed runs copied DESIGN. The task did not explain that this word was a placeholder.
- Best individual models: 14 / 18 tasks. Across all models: 15 / 18 tasks passed at least once.
- Recorded actions and failure analysis. Keep all original outcomes. A wording fix requires a new task version.

Comparison 2: the same tasks through four interfaces



- I ran four models on six tasks through four interfaces, giving 96 fresh attempts.
- Each attempt starts fresh, and the interface order is randomized within each model and task.
- This is a separate study with a \$25 allowance and no retries.

-
- Sol, Sonnet, Qwen and Grok • topic, thread reply, message edit, incident closeout, saved cleanup, decision record.
 - Seed 2042 • low reasoning • recent-four history • no guide • 40 actions • 180 seconds • \$1 ceiling per cell.

Interface comparison: 96 recorded attempts

Passed / attempted • six tasks per cell • click headers to sort

Model ↕	Accessibility ↕	Page JSON ↕	Pixels ↕	Actor API ↕	Allowance ↕
 GPT-6.1 Sol	<u>6 / 6</u> 0 incomplete • 0 blocked	<u>5 / 6</u> 0 incomplete • 1 blocked	<u>1 / 6</u> 0 incomplete • 5 blocked	<u>6 / 6</u> 0 incomplete • 0 blocked	\$3.9430
 Claude Sonnet 5.5	<u>4 / 6</u> 0 incomplete • 2 blocked	<u>4 / 6</u> 1 incomplete • 1 blocked	<u>2 / 6</u> 1 incomplete • 3 blocked	<u>5 / 6</u> 1 incomplete • 0 blocked	\$7.2784
 Qwen 3.8 Max	<u>5 / 6</u> 1 incomplete • 0 blocked	<u>4 / 6</u> 1 incomplete • 1 blocked	<u>0 / 6</u> 0 incomplete • 6 blocked	<u>5 / 6</u> 0 incomplete • 1 blocked	\$5.4829
 Grok 4.7	<u>3 / 6</u> 1 incomplete • 2 blocked	<u>3 / 6</u> 3 incomplete • 0 blocked	<u>0 / 6</u> 1 incomplete • 5 blocked	<u>5 / 6</u> 1 incomplete • 0 blocked	\$6.0573

[All traces and replays ↗](#) [Costs CSV ↓](#) [Paired results and timing ↗](#)

- API passed 21/24 attempts, accessibility passed 18/24, Page JSON passed 16/24, and pixels passed 3/24.
- The table separates incomplete tasks from attempts blocked by service failures or run limits.
- Click any pass count to inspect the model's six tasks, traces, and replays.

• Verified snapshot: 58 passed • 11 incomplete • 27 blocked • 0 not run. Usage estimate \$20.7180 + unresolved \$2.0435 = \$22.7616 of \$25. Updated 2026-10-05 07:42 UTC.

• Pixels use low-detail screenshots. Interfaces differ in visible information and action size, not just formatting.

These results do not isolate model ability

API AND ACCESSIBILITY



17 shared passes

Same model and task
Different information and actions

PIXEL INTERRUPTIONS



19 blocked runs

14 connection failures
3 output limits • 2 timeouts

STUDY SIZE



One attempt per condition

No repeat-run estimate
No general CUA ranking

- API actions can perform more work per step, so equal action limits do not create equivalent tests.
- Pixel runs used low-detail images, and keyboard shortcuts depended on macOS behavior.
- Service failures and run limits prevent some attempts from measuring task completion.

- 24 API/accessibility pairs: 17 both passed • 4 API only • 1 accessibility only • 2 neither passed. 5 pairs include a blocked attempt.
- On 17 shared passes, median accessibility-minus-API differences were +2 actions, +5.462 seconds, and +\$0.13008. These are selected successful pairs, not a general speed estimate.
- Elapsed time also includes shared-host activity. [Definitions, costs, and all matched pairs.](#)

Runs have separate data, but scaling is still untested



17.34 ms

Create session • median



1.50 ms

Reset state • median

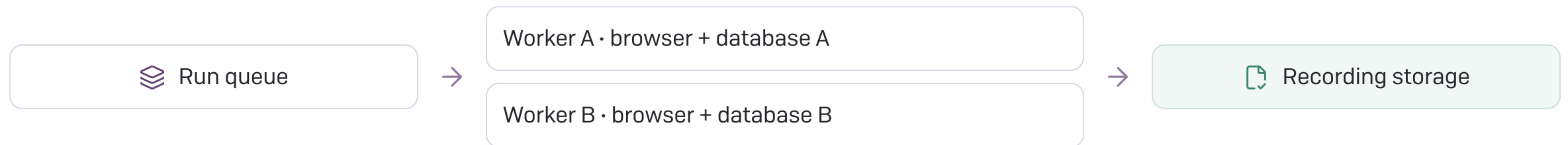


84.15 ms

Screenshot • median

- Each run has separate data, but these timings exclude model calls and shared-host concurrency.
- The proposed worker design still needs memory, throughput, and cleanup tests.

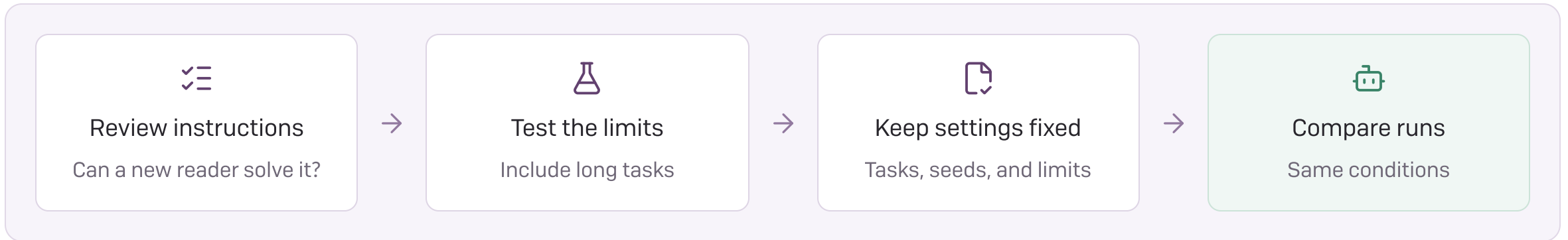
PROPOSED DESIGN



- Apple M5 Max • Node 24 • 100 sessions • 400 reads at concurrency 16 • 10 sequential browser sessions.
- Timing samples and test setup. The multi-worker design is proposed, not measured.

The next version needs validated tasks and repeated runs

- Ask someone outside the project to solve each task from its instructions.
- Compare their interpretation with the expected result.
- Set run limits with both short and long tasks.
- Use new task variants and repeat runs to measure reliability.



The current results describe these attempts. Repeated runs are needed to estimate reliability.

- [Detailed engineering review](#) · [Remaining validation work](#). These are next steps, not completed results.

Demo and design questions



01 · TRY

Use the workspace

Search, open a thread, edit



02 · COMPARE

Choose a model and task

All 306 attempts and costs



03 · REVIEW

Inspect the run

Actions, changes, result checks

- Which workflows should the next version test?
- Should the score include wrong actions that the agent later undoes?
- How many agents must run at once, and what should each run cost?

-
- Relay can reset, execute actions, and return a reward. No reinforcement learning training has been run.
 - [Code and setup](#) · [Requirements and remaining work](#).